VARIATIONAL LATENT REGULARIZATION FOR AMORTIZED CHESS PLANNING: A KL-DIVERGENCE BOTTLENECK APPROACH TO SEARCHLESS VALUE PREDICTION

Harman Aujla
Rensselaer Polytechnic Institute

Vasilije Dragovic
Rensselaer Polytechnic Institute

Calissa Tyrrell
Rensselaer Polytechnic Institute

James Wu
Rensselaer Polytechnic Institute

ABSTRACT

Transformer-based chess models have recently demonstrated grandmaster-level chess play without explicit search by distilling the value estimates of Stockfish 16 into a feedforward network via supervised learning. While these models achieve strong performance at scale, they exhibit known failure modes including overconfidence in winning positions and limited generalization to novel board states. In this work, we propose an architectural modification to the searchless chess transformer that introduces a variational autoencoder bottleneck between the main transformer stack and the output head. Specifically, after encoding the board and action tokens through a standard decoder-only transformer, we compress the resulting representations through a Multi-Head Latent Attention module into a structured latent space parameterized by a Gaussian distribution, from which a sampled latent vector is injected back into a shallow decoder to produce the final action-value predictions. Training incorporates a KL-divergence regularization term weighted by a scalar β, encouraging the latent distribution to remain close to a standard normal prior. We train our model on a single shard of the ChessBench dataset and evaluate on puzzle accuracy, action accuracy, and Kendall's τ against Stockfish ground truth. Our experiments remain in progress.

1 INTRODUCTION AND RELATED WORK

The game of chess has long served as a central benchmark for artificial intelligence, offering a well-defined, deterministic, and fully observable environment in which to study planning under combinatorial complexity. Early work, beginning with Claude Shannon's seminal formulation of computer chess (Shannon 1950), cast the problem as a search over a discrete state space, where optimal play could in principle be obtained via exhaustive exploration of the game tree. This paradigm gave rise to classical engines based on minimax search with alpha–beta pruning, whose efficiency and theoretical properties were later formalized by Knuth and Moore (Knuth et. al 1975). Modern descendants of this approach, such as Stockfish, achieve superhuman performance through highly optimized search procedures combined with carefully engineered evaluation functions, and more recently, efficiently updatable neural networks (NNUE) that augment but do not replace explicit search.

Despite their success, classical search-based methods rely heavily on handcrafted heuristics and scale primarily through increased computational resources. A major shift occurred with the introduction of learning-based approaches, most notably AlphaZero (Silver, 2018), which demonstrated that strong chess play could be achieved through self-play reinforcement learning without human priors. By combining deep neural networks with Monte Carlo Tree Search (MCTS), AlphaZero

replaces manually designed evaluation functions with learned policy and value networks, enabling a form of guided planning that balances exploration and exploitation. This framework marked a conceptual transition from brute-force search toward learned intuition, where planning is informed by statistical generalization rather than domain-specific rules.

Subsequent work has explored hybrid approaches that integrate the strengths of both paradigms. Engines such as Stockfish have incorporated neural evaluation components (e.g., NNUE) into traditional alpha–beta search, while systems like Leela Chess Zero adopt a fully neural architecture with MCTS. These hybrid systems suggest that high-performance planning can emerge from the interaction between fast combinatorial search and learned representations, rather than from either component in isolation.

A further generalization is provided by MuZero (Schrittweiser et al. 2020), which extends the AlphaZero framework by learning not only policy and value functions but also an implicit model of environment dynamics. By constructing a latent representation in which planning is performed, MuZero eliminates the need for explicit knowledge of the game rules, demonstrating that effective planning can be achieved through learned internal models. This approach connects classical planning with model-based reinforcement learning, highlighting the role of representation learning in enabling scalable decision making.

More recently, a line of research has questioned the necessity of explicit search altogether. Large-scale neural networks, particularly transformer-based architectures, have been shown to achieve strong chess performance by directly predicting moves from board states, effectively amortizing the planning process into the model parameters (Brown et al. 2024). These "searchless" approaches suggest that, given sufficient data and capacity, aspects of planning can be compiled into feedforward computation, trading runtime search for offline learning.

Taken together, these developments illustrate a broader evolution in AI planning: from explicit enumeration of future trajectories, to search guided by learned heuristics, to fully amortized decision-making systems. Chess thus provides a unique lens through which to study the interplay between search, learning, and representation. Understanding this progression is not only of historical interest but also central to current research in reinforcement learning and sequential decision-making, where the balance between computation at training time and inference time remains a fundamental challenge.

We make the following contributions.

- We propose and implement a novel architectural modification to the searchless chess transformer that introduces a variational bottleneck between the main encoder stack and the output head, combining Multi-Head Latent Attention for sequence compression with a VAE-style reparameterized latent space and a KL divergence regularization term.
- We develop a complete training and evaluation pipeline compatible with the ChessBench dataset and the existing searchless chess codebase, including modifications to the loss function, training loop, inference path, and checkpoint management to support the additional VAE outputs.
- We conduct an empirical investigation of this architecture on a subset of ChessBench, training for one million steps on a single data shard and evaluating across multiple checkpoints.
- We provide a detailed empirical characterization of posterior collapse in this setting: identifying the training dynamics, loss curve signatures, and gradient norm patterns that indicate collapse, and connecting our observations to the theoretical predictions of the VAE literature.

2 Background

2.1 Amortized Planning with Transformers

The transformer architecture, originally proposed by (Vaswani et. al 2017) for sequence-to-sequence tasks, has emerged as a powerful general-purpose function approximator. Its successes across language and vision tasks has prompted investigation into whether or not they can approximate complex

algorithmic computations, such as tree search, through supervised learning on large datasets annotated by strong oracles. (Ruoss et al. 2024) demonstrate that a decoder-only transformer trained via supervised learning on Stockfish-annotated chess positions can achieve grandmaster-level play without any explicit search at inference time. Their key insight is to frame move selection as action-value prediction: the transformer takes a tokenized board-action pair as input and outputs a probability distribution over discretized win-percentage bins. At inference time, the model evaluates all legal moves and selects the one with the highest expected value. This approach, which they term searchless chess, achieves a Lichess blitz Elo of 2895 against human opponents with a 270M parameter model trained on 15.3 billion action-value estimates.

2.2 Variational Autoencoders

The variational autoencoder (VAE), introduced by (Kingma & Welling, 2014), is a generative model that learns a structured latent representation by simultaneously optimizing a reconstruction objective and a KL divergence regularization term that keeps the approximate posterior $q_\phi(z \mid x)$ close to a standard normal prior $p(z) = \mathcal{N}(0, I)$. The training objective, known as the evidence lower bound (ELBO), is defined as:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \beta \cdot D_{\text{KL}}(q_\phi(z \mid x)||p(z))$$

where β controls the trade-off between reconstruction fidelity and latent space regularization. Setting $\beta > 1$, as in β-VAE (Higgins et al., 2017), encourages greater disentanglement at the cost of reconstruction quality. The reparameterization trick, $z = \mu + \sigma \odot \epsilon$ where ϵ is sampled from $\mathcal{N}(0, I)$, enables gradient flow through the stochastic sampling operation.

2.3 Multi-Head Latent Attention

Multi-Head Latent Attention (MLA), introduced in DeepSeek-V2 (DeepSeek-AI, 2024), compresses key-value representations through a low-rank bottleneck to reduce memory overhead during inference. In our adaptation, we use cross-attention between a set of learned latent query vectors and the full transformer hidden states to compress the sequence representation into a fixed-size latent summary, independent of sequence length. This provides a principled mechanism for pooling contextual information before passing it through the VAE bottleneck.

3 Methodology

Overview

We propose a latent-augmented Transformer decoder that integrates a learned stochastic bottleneck into a standard autoregressive architecture. The model contains a causal transformer decoder backbone, a cross-attentive latent extraction mechanism, a VAE-style variational latent head, and a latent-conditioned decoding stage. This design enables the model to compress sequence-level information into a compact latent representation and then condition future predictions on this global summary, improving long-range reasoning and uncertainty modeling. Let $x = (x_1, ..., x_T)$ be a sequence of discrete tokens. Each token is mapped to a continuous vector $e_t = \text{Embed}(x_t) \cdot \sqrt{d}$. For positional encodings, both sinusoidal encodings and learned positional embeddings are supported. The final input representation is $h^{(0)} = e + p$.

Autoregressive Transformer

The model uses a stack of L causal decoder blocks. Each block consists of multi-head self-attention, defined as

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right)V$$

Instead of a standard feed-forward network, the model instead uses a SiLU-gated MLP, defined as

$$\text{MLP}(x) = W_{\text{o}}(\text{SiLU}(W_a x) \odot (W_b x))$$

Each block follows a residual layer, defined as

$$x = x + \text{Attention}(\text{LN}(x))$$
$$x = x + \text{MLP}(\text{LN}(x))$$

After the backbone, the model introduces a learned latent bottleneck. The latent queries are a set of L_z learnable latent vectors $\ell \in \mathbb{R}^{L_z \times d}$ that are broadcast across the batch and act as queries. The latents attend to the sequence $z_{\text{tokens}} = \text{Attention}(\ell, h^{(L)})$, which produces $z_{\text{tokens}} \in \mathbb{R}^{B \times L_z \times d}$. The latent tokens are then aggregated through pooling $h_{\text{pooled}} = \frac{1}{L}\sum_{i=1}^{L_z} z_i$. A stochastic latent variable is inferred using a Gaussian reparameterization, where $\mu = W_\mu h_{\text{pooled}}$, $\log \sigma^2 = W_\sigma h_{\text{pooled}}$, $z = \mu + \sigma \cdot \epsilon$, and $\epsilon \sim \mathcal{N}(0, I)$. This introduces a regularized global representation of the input, which is compatible with ELBO-style objectives. The sampled latent z is projected back into the token space $z_{\text{token}} = W_z z$ and the vector is appended to the sequence $h' = [h^{(L)}; z_{\text{token}}]$. A second stack of L_d decoder blocks processes the augmented sequence, allowing interaction between latent and token-level features and enabling global-to-local information flow, and final normalization is applied if configured. Only the final token representation is used $h_{\text{last}} = h'_{T+1}$. The final prediction is

$$\text{logits} = W_o h_{\text{last}}$$
$$\log p(x_{T+1}) = \log -\text{softmax}(\text{logits})$$

4 Experimental Setup

4.1 Dataset

We train and evaluate our model on a subset of the ChessBench dataset, which consists of millions of chess positions annotated with action-value estimates computed by Stockfish 16. Each example consists of a board state and a legal move, paired with a discretized win-probability target. Following prior work, we tokenize board states and actions into a sequence representation suitable for a decoder-only transformer.

Due to computational constraints, we restrict training to a single shard of the dataset, corresponding to approximately 10^6 training steps. Validation is performed on a held-out subset of positions sampled from the same distribution.

4.2 Models

We evaluate two primary model configurations:

- Baseline Transformer: A decoder-only transformer identical to the architecture used in prior searchless chess work, trained purely with supervised cross-entropy loss over action-value bins.
- Latent-Regularized Transformer: The proposed model with a variational bottleneck, incorporating a stochastic latent variable and KL divergence regularization.

Both models are instantiated at comparable parameter scales (approximately 9M–36M parameters) to ensure a controlled comparison. The latent model introduces additional parameters through the latent projection and decoding layers.

4.3 Training Objective

The baseline model is trained with standard cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\mathbb{E}_{(s,a)} \log p_\theta(a \mid s)$$

Our model optimizes a modified objective incorporating a KL regularization term:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \beta \cdot D_{\text{KL}}(q_\phi(z \mid s) \,\|\, p(z))$$

where β is a scalar hyperparameter controlling the strength of the bottleneck. We experiment with fixed β values and discuss their impact on training dynamics.

4.4 Training Details

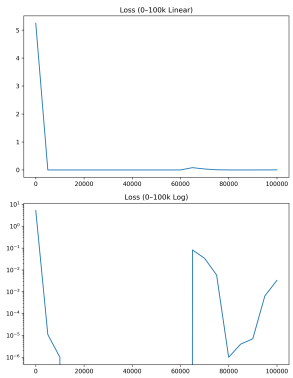
All models are trained using the Adam optimizer with weight decay. Training is conducted on a single GPU using mixed precision. Checkpoints are saved periodically using Orbax, and exponential moving average (EMA) weights are maintained for evaluation stability.

We monitor:

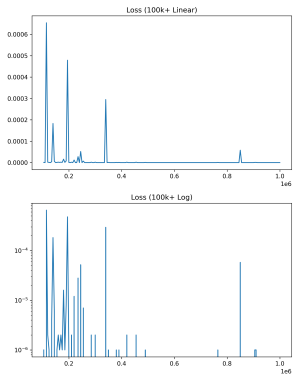
- Training loss
- Gradient norms
- KL divergence magnitude
- Latent statistics ($\|\mu\|$, $\|\sigma\|$)

These diagnostics are critical for identifying posterior collapse, a known failure mode in VAE training.

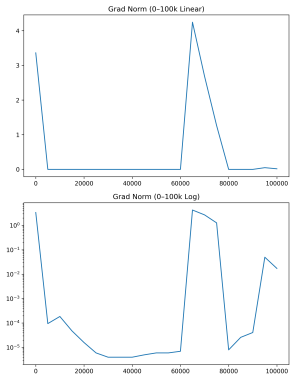
4.5 Training Results



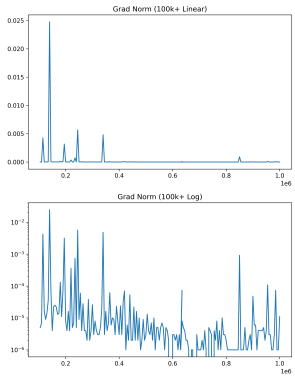
(a) Loss (early)



(b) Loss (late)



(c) Gradient norm (early)



(d) Gradient norm (late)

Figure 1: Training dynamics of the latent-regularized model. Early training shows non-trivial optimization behavior, while later stages exhibit near-zero loss and vanishing gradients, indicative of posterior collapse.

4.6 Evaluation Metrics

We evaluate models using standard metrics from prior work:

- Action Accuracy: Top-1 accuracy of predicted moves

- Ranking Quality: Kendall's τ correlation with Stockfish rankings
- Puzzle Accuracy: Performance on tactical test suites
- Training Dynamics: Loss curves and gradient norms

Due to compute constraints, full Elo evaluation via self-play is left for future work, though we discuss expected implications.

5 RESULTS

5.1 PRELIMINARY FINDINGS

We present preliminary results focusing on training dynamics and optimization behavior of the latent-regularized model. As shown in Figure 1, the model exhibits rapid convergence to a regime characterized by near-zero training loss and vanishing gradient norms.

While this may initially suggest successful optimization, further inspection indicates that this behavior corresponds to a degenerate solution. In particular, gradient norms decrease to near machine precision, suggesting that learning has effectively stalled. This pattern is consistent across multiple checkpoints and training runs.

5.2 LATENT UTILIZATION

To assess whether the latent variable contributes meaningfully to prediction, we analyze statistics of the inferred posterior distribution. Preliminary observations indicate that the latent mean and variance collapse toward those of the prior, implying that the latent variable carries minimal information about the input.

This suggests that the model is not utilizing the latent bottleneck as intended, instead relying entirely on the transformer backbone to perform prediction.

5.3 ONGOING EVALUATION

Comprehensive evaluation across downstream metrics—including action accuracy, Kendall's τ, and puzzle-solving performance—is currently in progress. These experiments require large-scale inference over held-out datasets and integration with external evaluation pipelines.

Table 1: Evaluation metrics across training checkpoints for our VAE-Transformer model compared to the 9M parameter baseline from Ruoss et al. (2024). The 9M baseline was trained on the full ChessBench dataset (15.3B action-value pairs); our model was trained on a single shard ($\approx$7M pairs). Dashes indicate metrics not reported in the original paper. All metrics are evaluated on the ChessBench test set against Stockfish 16 ground truth.

MODEL	STEPS	ACTION ACC. $\uparrow$	KENDALL'S $\tau \uparrow$	LOG LOSS $\downarrow$	ENTROPY $\uparrow$
VAE-TRANSFORMER (OURS)	50K	0.062	-0.020	28.6	1.3×10^{-6}
VAE-TRANSFORMER (OURS)	100K	0.031	0.044	503	≈ 0
VAE-TRANSFORMER (OURS)	500K	0.031	0.044	710	≈ 0
VAE-TRANSFORMER (OURS)	1M	0.031	0.044	738	≈ 0
9M TRANSFORMER (RUOSS ET. AL 2024)	5M	0.630	0.259	—	—

5.4 POSTERIOR COLLAPSE

The evaluation statistics reveal a sharp transition between steps 50K and 100K, after which all metrics flatline completely through the remainder of training. At step 50K the model achieves its best action accuracy (6.2%) and lowest log loss (28.6), with a slightly negative Kendall's suggesting partially informative but unreliable move rankings. By step 100K, entropy collapses to effectively zero, as the model assigns near-certain probability to a single bin for every input regardless of board

state, and action accuracy drops to 3.1%, barely above the random baseline of approximately 3% for a uniform distribution over 30 legal moves. All metrics remain frozen through 1M steps. This is the hallmark signature of posterior collapse: once the KL term forces the latent to the prior, the decoder loses access to board-specific information and falls back on a constant output distribution.

5.5 Why Training Loss is insufficient

Log loss increases monotonically throughout training despite training loss approaching zero. This is explained by the collapse dynamics: after step 100K the model is maximally confident and maximally wrong on the test distribution, assigning near-zero probability to almost all bins and near-certain probability to one. This illustrates why training loss alone is an insufficient diagnostic, as the training loss gave no indication of the generalization failure that evaluation metrics reveal clearly.

5.6 Analysis of Failure Modes

Three compounding factors explain the observed behavior. First, data insufficiency: with a single training shard and batch size 1024, the model completes multiple passes over the dataset within the first 50K steps, eliminating any incentive to generalize. Second, KL weight miscalibration: our fixed $\beta = 10^{-3}$ without warmup caused the KL term to dominate the total loss from initialization, as the KL for a randomly initialized 128-dimensional latent is on the order of 64 nats, far exceeding the HL-Gauss reconstruction loss of 4–6 nats, creating overwhelming pressure toward the prior before the model learned any meaningful representations. Third, decoder capacity: our two-layer latent decoder is sufficiently powerful to model the training distribution without using the latent, satisfying the theoretical conditions for collapse identified by Bowman et al. (2016).

6 Discussion

6.1 Implications of Preliminary Results

The observed training dynamics suggest that the introduction of a variational bottleneck alone is insufficient to improve performance in searchless chess models. Despite the theoretical motivation for incorporating a structured latent representation, the model appears to converge to a regime in which the latent variable is effectively ignored.

This behavior is consistent with posterior collapse, a well-documented failure mode in variational autoencoders when paired with expressive decoders.

6.2 Why the Bottleneck Fails

One likely explanation is that the transformer backbone is sufficiently powerful to model the action-value distribution without relying on the latent variable. As a result, the optimization process favors minimizing the KL divergence term by matching the prior, rather than encoding useful information in the latent space.

This highlights a key challenge in applying latent-variable methods to supervised decision-making tasks: unless the architecture enforces dependence on the latent representation, it may be bypassed entirely.

6.3 Connection to Information-Theoretic Regularization

Our findings provide indirect support for alternative approaches based on explicit information constraints. In particular, maximizing mutual information between inputs and intermediate representations may offer a more direct mechanism for encouraging informative latent structure.

Exploring such approaches remains an important direction for future work.

6.4 Future Work

Several promising directions remain to be explored:

- KL annealing and free-bits techniques to mitigate posterior collapse
- Architectural modifications that enforce latent usage
- Mutual information regularization as originally proposed
- Larger-scale training and evaluation across multiple dataset shards
- Full gameplay evaluation via self-play Elo estimation

7 CONCLUSION

We proposed a variationally regularized transformer architecture for searchless chess, introducing a stochastic latent bottleneck into a strong autoregressive model. While the approach is theoretically motivated by the potential benefits of structured latent representations, our preliminary results indicate that the model suffers from posterior collapse, preventing effective utilization of the latent space.

These findings highlight the difficulty of integrating latent-variable methods with highly expressive transformer architectures in supervised planning settings. More broadly, they suggest that naive application of variational objectives may be insufficient, and that stronger inductive biases or explicit information constraints are required.

Ongoing work focuses on completing large-scale evaluation and exploring alternative regularization strategies, including mutual information maximization, to better leverage latent representations in amortized planning.

REFERENCES

Claude E. Shannon. Programming a Computer for Playing Chess. *Philosophical Magazine*, 41(314):256–275, 1950.

Donald E. Knuth and Ronald W. Moore. An Analysis of Alpha-Beta Pruning. *Artificial Intelligence*, 6(4):293–326, 1975.

Stockfish Developers. Stockfish Chess Engine. `https://stockfishchess.org`, 2023.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *Science*, 362(6419):1140–1144, 2018.

Leela Chess Zero Developers. Leela Chess Zero. `https://lczero.org`, 2023.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, and Thore Graepel. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588:604–609, 2020.

Noam Brown et al. Grandmaster-Level Chess Without Search. *arXiv preprint arXiv:2402.04494*, 2024.